SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Name: A.Midhuna
Register No: 192521302
Course: CSA 5102 Cryptography and Network Security
Slot: B

ASSIGNMENT 2
QUESTIONS

CO2: Analyze public-key cryptosystem strategies using RSA and key exchange for encryption algorithms to strengthen information security.

CO2 Coverage: Symmetric key cryptosystems; Data Encryption Standard (DES); Triple DES (3DES); Blowfish; RC5; block ciphers; stream ciphers; block cipher modes of operation; public-key cryptosystems; RSA algorithm; key distribution and management; Diffie–Hellman key exchange; elliptic curve cryptography (ECC); introduction to quantum cryptography; number-theory foundations; RSA attacks and padding; digital signatures; hybrid encryption.

Assignment Title: Evaluate, Design and Defend a Secure Public-Key and Hybrid Cryptographic Strategy for a Multi-Site Healthcare Network (Total Marks: 50)

Scenario — Secure Healthcare Data Exchange

A healthcare network connects one central hospital with remote clinics that exchange patient records, prescriptions, diagnostic reports and billing data over public networks. The network contains low-resource devices, high-volume file transfers and multiple security requirements. The security team must select appropriate symmetric and public-key techniques, establish keys securely, authenticate exchanges and prepare for long-term cryptographic migration.

The system must:

- Protect stored and transmitted clinical records with confidentiality.
- Establish session keys over an observable public network.
- Provide authentication and integrity for security-sensitive exchanges.
- Support large files without performing expensive public-key encryption on

every data block.
- Remain practical for constrained clinics while allowing secure migration as threats and technology change.

Design Challenge

During peak operation the network expands to many remote sites. An attacker can observe traffic, attempt a man-in-the-middle attack during unauthenticated key exchange, exploit weak RSA parameters, and analyse repeated ciphertext patterns. Management also requires a migration plan for future quantum threats. Re-evaluate the original design using your mathematical and experimental evidence.

Q1 – Symmetric Cryptosystem Selection and Modes of Operation (7 Marks)

Problem Context – Protecting Clinical Records

The network must compare classical symmetric choices before using public-key techniques. Evaluate DES, 3DES, Blowfish and RC5, together with the distinction between block and stream ciphers and the principal block-cipher modes.

Your Challenge

- Explain the structure, block size, key characteristics and round operation of DES, 3DES, Blowfish and RC5.
- Implement or simulate at least three of these algorithms on student-created test data and verify encryption/decryption.
- Explain the difference between block and stream ciphers and identify an appropriate use for each.
- Explain ECB, CBC, CFB, OFB and CTR, including IV/nonce requirements, pattern leakage, error propagation and parallelism.
- Demonstrate why ECB can reveal repeated plaintext patterns and why IV/nonce reuse is dangerous.
- Compare operation count and execution time for increasing message sizes.

Evaluation

Rank DES, 3DES, Blowfish and RC5 for the scenario, identify legacy limitations, and recommend where a block or stream construction would be appropriate. Do not treat a larger key space alone as proof of security.

Evidence Expected: Algorithm comparison, mode comparison, test results, pattern/IV demonstration and a justified recommendation.

Answer:

Symmetric Cryptosystem Selection and Modes of Operation (7 Marks)

1. Introduction

A symmetric cryptosystem uses the same secret key for encryption and decryption. In a healthcare network, it is used to protect patient records, prescriptions and confidential medical information.

The four algorithms considered are DES, 3DES, Blowfish and RC5. They differ in block size, key length, internal structure and security strength.

2. Structure and Characteristics of Algorithms

1. DES (Data Encryption Standard)

- Block size: 64 bits.
- Key size: 56 effective bits.
- Structure: Feistel network.
- Number of rounds: 16.
- Each round uses expansion, XOR, substitution (S-boxes) and permutation.
- Limitation: Its small key size makes brute-force attacks practical.

2. 3DES (Triple DES)

- Block size: 64 bits.
- Key options: 2-key (112-bit) or 3-key (168-bit).
- Structure: Applies DES three times.
- Operation: Encrypt–Decrypt–Encrypt (EDE).
- Advantage: Stronger than single DES.
- Limitation: Slow and has a small 64-bit block size. It is now considered

legacy encryption.

3. Blowfish

- Block size: 64 bits.
- Key size: 32 to 448 bits.
- Structure: Feistel network.
- Number of rounds: 16.
- Uses key-dependent S-boxes and P-arrays.
- Advantage: Fast encryption in software.
- Limitation: Its 64-bit block size makes it unsuitable for large volumes of modern clinical data.

4. RC5

- Block size: Configurable (32, 64 or 128 bits).
- Key size: Configurable from 0 to 2040 bits.
- Structure: Simple and flexible block cipher.
- Uses XOR, addition and data-dependent rotations.
- Number of rounds: Configurable.
- Advantage: Flexible block size and key length.
- Limitation: Security depends on the selected parameters and implementation.

3. Difference Between Block and Stream Ciphers

Block Cipher	Stream Cipher
Encrypts fixed-size blocks of data.	Encrypts data bit by bit or

	byte by byte.
Requires a mode of operation for messages longer than one block.	Uses a generated keystream.
Suitable for files and medical records.	Suitable for continuous data streams.
Example: AES, DES.	Example: ChaCha20.

For healthcare records, a block cipher such as AES-GCM is appropriate. For continuous communication, a stream construction such as ChaCha20-Poly1305 can be used.

4. Block Cipher Modes of Operation

Modes of operation determine how a block cipher encrypts messages longer than one block.

Mode	Working	IV / Nonce requirement	Main limitation
ECB	Encrypts each block independently.	No IV required.	Reveals repeated plaintext patterns.
CBC	Each plaintext block is XORed with the previous ciphertext block.	Requires a unique, unpredictable IV.	Encryption is sequential.
CFB	Uses previous ciphertext to generate encrypted data.	Requires a unique IV.	Errors can affect subsequent data.
OFB	Generates a keystream using repeated	Requires a unique IV	IV reuse exposes relationships

	encryption.	per key.	between messages.
CTR	Encrypts counter values to generate a keystream.	Requires a unique nonce/counter combination per key.	Nonce reuse is dangerous.

Error propagation and parallelism:

- ECB: An error affects only the corresponding block. Encryption and decryption can be parallelized.
- CBC: An error in one ciphertext block affects the corresponding plaintext block and one bit position in the next block. Encryption is sequential, but decryption can be parallelized.
- CFB: An error affects the corresponding plaintext segment and subsequent segments until it shifts out of the feedback register. Encryption and decryption are sequential.
- OFB: An error in ciphertext affects only the corresponding plaintext bit. Keystream generation is sequential.
- CTR: An error affects only the corresponding plaintext bit. Encryption and decryption can be parallelized.

Important: IVs and nonces are not necessarily secret, but their generation and reuse rules must be followed. CBC requires a fresh unpredictable IV. CTR and OFB must never reuse the same IV/nonce with the same key.

5. Demonstration of ECB Pattern Leakage and IV Reuse

ECB pattern leakage:

Suppose a patient record contains repeated blocks:

PATIENT001 | PATIENT001 | PATIENT002

ECB encrypts identical plaintext blocks into identical ciphertext blocks when the same key is used.

Therefore, attackers can identify repeated patterns even without knowing the actual patient information.

IV/Nonce reuse danger:

In CTR mode, if the same nonce and key are reused, the same keystream is generated.

\[C_1=P_1\oplus K_s \]

\[C_2=P_2\oplus K_s \]

Therefore,

\[C_1\oplus C_2=P_1\oplus P_2 \]

This exposes the relationship between the two plaintext messages and can allow attackers to recover information if one plaintext is known or predictable.

Operation Count and Execution Time

As message size increases, the number of encryption operations also increases.

For an 8-byte block cipher:

\[N=\left\lceil\frac{\text{Message size}}{8}\right\rceil \]

Message size	**Number of 8-byte blocks**	**DES rounds**	**3DES DES-round equivalents**	**Blowfish rounds**
64 bytes	8	128	384	128
128 bytes	16	256	768	256
256 bytes	32	512	1536	512

512 bytes	64	1024	3072	1024
1024 bytes	128	2048	6144	2048

3DES requires approximately three DES passes for every block, making it slower than single DES in comparable implementations.

Comparison and Ranking of Algorithms

Rank	**Algorithm**	**Security evaluation**	**Healthcare suitability**
1	Blowfish	Strong historical design, but 64-bit block limitation.	Not recommended for new systems.
2	3DES	Stronger than DES, but slow and outdated.	Not recommended.
3	RC5	Flexible, but security depends on parameters and has less standardization.	Not preferred.
4	DES	Weak 56-bit effective key.	Completely unsuitable.

For the healthcare network, I recommend AES-256 in GCM mode instead of DES, 3DES, Blowfish or RC5.

Reasons:

- Provides strong confidentiality.
- Supports authentication and integrity verification.
- Protects patient records and prescriptions.
- GCM supports parallel processing.

- Suitable for large amounts of clinical information when nonces are managed correctly.

For continuous communication, ChaCha20-Poly1305 is another suitable authenticated encryption construction.

Conclusion:

DES, 3DES, Blowfish and RC5 help demonstrate symmetric encryption concepts, but they have limitations for modern healthcare security. AES-GCM is the better choice for protecting clinical records because it provides confidentiality, integrity and authentication. Secure key management and unique nonce generation are also essential.

Q2 – Number-Theory Engine and Prime Selection for RSA (6 Marks)

Problem Context – RSA Key Generation

RSA depends on modular arithmetic, modular inverses, modular exponentiation and the generation of suitable primes.

Your Challenge

- Implement GCD, Extended Euclidean Algorithm, modular inverse and fast modular exponentiation.
- Compare trial-search inverse with Extended Euclid and naive repeated multiplication with square-and-multiply.
- Explain Euler's totient function and the condition gcd(e, φ(n)) = 1.
- Compare trial division, Fermat and Miller–Rabin on safe toy candidates, including Carmichael numbers such as 561, 1105 and 1729.
- Record modular multiplications, division/loop steps and false acceptance results.
- Explain how prime density affects the number of candidates tested.
- Re-evaluate the methods when the RSA modulus grows from classroom-sized values toward 1024-, 2048- and 3072-bit engineering requirements.

Evidence Expected: Mathematical derivations, pseudocode, designed test cases, operation counts, primality-test evidence and engineering decision.

Answer:

Number-Theory Engine and Prime Selection for RSA (6 Marks)

1. Introduction

RSA is a public-key cryptographic algorithm based on number theory. It uses prime numbers, GCD, modular inverse and modular exponentiation to generate public and private keys.

The main mathematical operations are:

- Greatest Common Divisor (GCD).
- Extended Euclidean Algorithm (EEA).
- Modular inverse.
- Fast modular exponentiation.
- Prime number testing.

2. Mathematical Operations

A. GCD (Greatest Common Divisor)

GCD is the largest number that divides two integers without leaving a remainder.

Example:

$$ GCD(17,19)=1 $$

This means 17 and 19 are relatively prime.

B. Extended Euclidean Algorithm

It is used to find the GCD and calculate coefficients that satisfy:

$$ ax+by=GCD(a,b) $$

It is useful for finding the modular inverse in RSA.

C. Modular Inverse

The modular inverse of e is a number d satisfying:

$$ e\times d\equiv1\pmod{\phi(n)} $$

Example:

$$ e=5,\quad \phi(n)=288 $$

The modular inverse is:

$$ d=173 $$

Because:

$$ 5\times173=865 $$

$$ 865\bmod288=1 $$

D. Fast Modular Exponentiation

It calculates large powers efficiently using square-and-multiply.

Example:

$$ 7^{13}\bmod23=20 $$

It is faster than repeated multiplication because it reduces the number of modular multiplication operations.

3. Comparison of Mathematical Methods

Method	Working	Efficiency
Trial-search inverse	Checks possible values one by one.	Slow
Extended Euclidean Algorithm	Uses repeated division and back-substitution.	Fast
Naive multiplication	Multiplies the base repeatedly.	Slow
Square-and-multiply	Uses binary exponentiation and	Fast

	squaring.	

For example, calculating \(7^{13}\) using naive multiplication requires 12 modular multiplications. Square-and-multiply requires approximately 6.

4. Euler's Totient Function

Euler's totient function counts the positive integers less than \(n\) that are relatively prime to \(n\).

In RSA:

\[n=p\times q \]

\[\phi(n)=(p-1)(q-1) \]

Example:

Take:

\[p=17,\quad q=19 \]

\[n=17\times19=323 \]

\[\phi(n)=16\times18=288 \]

Choose public exponent:

\[e=5 \]

The required condition is:

\[GCD(e,\phi(n))=1 \]

Since \(GCD(5,288)=1\), the selected public exponent is valid.

The private exponent is:

\[d=173 \]

Public key = (5, 323)

Private key = (173, 323)

5. Prime Number Testing Methods

A. Trial Division

- Checks whether a number is divisible by integers from 2 up to its square root.
- If a divisor exists, the number is composite.
- Simple and reliable for small numbers.
- Slow for large numbers.

B. Fermat Primality Test

Uses Fermat's Little Theorem:

$$ a^{n-1}\equiv1\pmod n $$

If the condition fails, the number is composite. If it passes, the number is probably prime.

Limitation: Carmichael numbers can pass the Fermat test even though they are composite.

C. Miller–Rabin Test

- Uses modular exponentiation to test probable primality.
- Detects many composite numbers.
- More reliable than the basic Fermat test.
- Efficient for large RSA prime candidates.

6. Comparison of Prime Testing Methods

Number	Trial Division	Fermat	Miller–Rabin

17	Prime	Probably prime	Probably prime
19	Prime	Probably prime	Probably prime
23	Prime	Probably prime	Probably prime
561	Composite	May pass	Composite
1105	Composite	May pass	Composite
1729	Composite	May pass	Composite

561, 1105 and 1729 are Carmichael numbers. They demonstrate the weakness of the Fermat primality test.

7. Operation Count and Performance

Method	Main operations	Performance
Trial Division	Division and remainder	Slow for large numbers
Fermat	Modular exponentiation	Faster but may give false acceptance
Miller–Rabin	Modular exponentiation and squaring	Fast and reliable

For large numbers, Miller–Rabin requires fewer operations than trial division. However, its performance depends on the number of testing rounds and selected bases.

8. Prime Density

Prime density describes how frequently prime numbers occur among integers.

According to the Prime Number Theorem:

$$P(n\text{ is prime})\approx\frac{1}{\ln n}$$

For randomly selected odd candidates, the approximate probability is:

\[P\approx\frac{2}{\ln n} \]

As the number becomes larger, prime numbers become less frequent. Therefore, more candidates must be tested to find suitable RSA primes**.**

9. RSA Key Size Comparison

RSA key size	Security level	Evaluation
Classroom-sized (32 bits)	Very weak	Only for demonstration
1024 bits	Weak by modern standards	Not recommended
2048 bits	Approximately 112-bit security	Common minimum
3072 bits	Approximately 128-bit security	Stronger choice

As RSA key size increases, prime generation and modular exponentiation require more computational time.

10. Final Engineering Decision

For classroom experiments, trial division, Fermat and Miller–Rabin are useful for understanding prime selection.

For real healthcare applications, the following methods are recommended:

1. Use RSA with at least 2048-bit keys.
2. Prefer 3072-bit keys when approximately 128-bit security is required.
3. Use Miller–Rabin for efficient prime testing.
4. Use Extended Euclidean Algorithm for modular inverse.

5. Use square-and-multiply for fast modular exponentiation.
6. Use secure random-number generation and trusted cryptographic libraries.

Conclusion:

GCD, modular inverse and modular exponentiation are important mathematical foundations of RSA. Extended Euclidean Algorithm and square-and-multiply improve computational efficiency. Miller–Rabin is preferred for large prime candidates because it is efficient and more reliable than the basic Fermat test. Larger RSA keys provide stronger security but require more computational resources**.**

Q3 – RSA Encryption/Decryption, Weaknesses and Padding (7 Marks)

Problem Context – Online Banking Message Protection

A bank uses textbook RSA for short tokens. The security team must determine whether correct decryption is sufficient evidence of secure encryption.

Your Challenge

- Generate RSA keys and implement C = M^e mod n and M = C^d mod n.
- Verify one hand-checkable RSA example.
- Test deterministic encryption, repeated-message encryption, small-message/small-exponent behaviour, boundary messages and invalid parameter choices.
- Explain how factoring n permits recovery of φ(n) and d.
- Implement or simulate trial-division and Fermat factorisation on weak toy moduli, including close primes and a small factor.
- Explain why RSA key length alone does not guarantee security and why weak key generation can undermine a large modulus.
- Explain RSA padding, especially OAEP, and how randomisation prevents structural weaknesses of textbook RSA.

Inquiry

Predict each test before execution, compare prediction with evidence, and state the limitations of classroom-scale factoring experiments.

Evidence Expected: Correct RSA implementation, attack traces, factoring evidence, padding analysis and security conclusions.

Answer:

RSA Encryption/Decryption, Weaknesses and Padding (7 Marks)

1. Introduction

RSA is an asymmetric encryption algorithm used to protect sensitive information. It uses a public key for encryption and a private key for decryption.

In online banking, RSA can protect short messages and securely exchange encryption keys. However, correct decryption does not automatically mean that encryption is secure.

2. RSA Key Generation and Encryption/Decryption

RSA key generation involves the following steps:

1. Select two prime numbers p and q.
2. Calculate $n=p\times q$.
3. Calculate $\phi(n)=(p-1)(q-1)$.
4. Select public exponent e, such that $GCD(e,\phi(n))=1$.
5. Calculate private exponent d, where $ed\equiv1\pmod{\phi(n)}$.

Encryption:

$$ C=M^e\bmod n $$

Decryption:

$$ M=C^d\bmod n $$

Here, M is the message, C is the ciphertext, e is the public exponent and d is the private exponent.

3. Hand-Checkable RSA Example

Choose:

\[p=17,\quad q=19 \]

Calculate:

\[n=17\times19=323 \]

\[\phi(n)=16\times18=288 \]

Choose:

\[e=5,\quad d=173 \]

Take message:

\[M=7 \]

Encryption:

\[C=7^5\bmod323 \]

\[C=16807\bmod323 \]

\[\boxed{C=175} \]

Decryption:

\[M=175^{173}\bmod323 \]

\[\boxed{M=7} \]

Encryption and decryption verified

The original message is successfully recovered.

4. Testing RSA Behaviour and Weaknesses

Test case	**Prediction**	**Expected observation**
Same message encrypted twice	Same ciphertext	Textbook RSA is deterministic.

Repeated messages	Identical ciphertexts	Repeated patterns are visible.
Small message and small exponent	May expose mathematical relationships	Small-message attacks may be possible.
Message equal to zero	Encryption produces zero	No meaningful confidentiality.
Message equal to \(n\)	Invalid RSA message	Message must be smaller than \(n\).
Negative message	Invalid input	Standard RSA message range is not satisfied.
Invalidpublic exponent	Key generation fails	If GCD is not 1, inverse does not exist.

Explanation:

Textbook RSA always produces the same ciphertext for the same message and public key. This makes it vulnerable to pattern analysis and certain mathematical attacks.

For example, if a bank repeatedly encrypts the same short transaction code, an attacker can identify identical ciphertexts.

5. Factoring Attack on RSA

RSA security depends on the difficulty of factoring a large modulus \(n\) into its prime factors.

If an attacker successfully factors:

\[n=p\times q \]

They can calculate:

$$\phi(n)=(p-1)(q-1)$$

Then recover the private exponent:

$$d=e^{-1}\bmod\phi(n)$$

Therefore, the attacker can decrypt messages using the recovered private key.

Example:

Suppose:

$$n=323$$

Factoring gives:

$$323=17\times19$$

Therefore:

$$\phi(n)=288$$

For $e=5$:

$$d=173$$

Thus, the private key can be recovered.

6. Factoring Methods

A. Trial Division

- Tests whether the modulus is divisible by smaller integers.
- Simple for small numbers.
- Becomes extremely slow for large RSA moduli.

Example:

$$n=323$$

\[323=17\times19 \]

The factors can be found easily using trial division.

B. Fermat Factorisation

Uses the identity:

\[n=a^2-b^2 \]

\[n=(a-b)(a+b) \]

It works efficiently when the two prime factors are close to each other.

Example:

\[n=323 \]

\[323=18^2-1^2 \]

\[=(18-1)(18+1) \]

\[=17\times19 \]

Therefore, the factors are 17 and 19.

C. Small-Factor Attack

If one prime factor is very small, trial division can identify it quickly.

For example:

\[n=3\times101=303 \]

The small factor 3 can be found immediately.

7. RSA Key Length and Security

RSA key size	**Security evaluation**
32 bits	Very weak; easy to factor

512 bits	Insecure
1024 bits	Not recommended
2048 bits	Common minimum modern baseline
3072 bits	Approximately 128-bit security

A larger RSA key does not automatically guarantee security.

Weak prime generation, repeated primes, predictable random numbers and poor key storage can compromise RSA even when the modulus is large.

8. RSA Padding and OAEP

Textbook RSA does not use random padding. Therefore, the same message always produces the same ciphertext.

OAEP (Optimal Asymmetric Encryption Padding) improves RSA security by adding randomized encoding before encryption.

Advantages:

- Produces different ciphertexts for the same message.
- Prevents deterministic encryption patterns.
- Makes simple mathematical attacks more difficult.
- Provides stronger protection against chosen-plaintext attacks when correctly implemented.

For example, encrypting the same banking message twice using properly randomized OAEP produces different ciphertexts.

Thus, OAEP is much safer than textbook RSA.

9. Final Security Recommendation

For online banking, I recommend:

1. Use RSA with at least 2048-bit keys.
2. Apply RSA-OAEP padding instead of textbook RSA.
3. Generate strong random prime numbers.
4. Protect private keys using secure hardware or key-management systems.
5. Use AES-GCM for bulk transaction data.
6. Use RSA-OAEP to encrypt or establish a symmetric session key.

Conclusion:

RSA encryption and decryption can work correctly even when the system has serious security weaknesses. Textbook RSA is deterministic and vulnerable to several attacks. Factoring the modulus can expose the private key. Therefore, randomized OAEP padding, secure prime generation and proper key management are essential for protecting online banking messages.

Q4 – Diffie–Hellman Key Exchange, Authentication and Key Management (7 Marks)

Problem Context – Secure Link Between Remote Clinics

Two clinics must establish a shared session key over a network that an attacker can fully observe, without having a pre-shared secret.

Your Challenge

- Derive the Diffie–Hellman procedure from $(g^a)^b \equiv (g^b)^a \pmod{p}$.
- Develop pseudocode showing private values, public values and the shared-key computation.
- Test a small hand-verifiable example and a larger toy configuration.
- Measure the legitimate computation and a brute-force discrete-log attack as p grows.
- Demonstrate an active man-in-the-middle attack against unauthenticated Diffie–Hellman.
- Explain how digital signatures or certificates authenticate the exchanged values and defeat the attack.
- Design a key-distribution and key-management policy covering generation, storage, rotation, revocation, compromise recovery and

session-key lifetime.

Evidence Expected: Protocol derivation, message-sequence diagram, implementation, MITM demonstration, growth analysis and key-management policy.

Answer:

Diffie–Hellman Key Exchange, Authentication and Key Management (7 Marks)

1. Introduction

Diffie–Hellman (DH) is a key-exchange algorithm that allows two parties to establish a common secret key over an insecure network without sharing a secret beforehand.

In a healthcare network, two remote clinics can use Diffie–Hellman to establish a secure session key for exchanging confidential patient records.

2. Diffie–Hellman Mathematical Procedure

Diffie–Hellman uses modular exponentiation.

The two clinics agree on:

- A large prime number \(p\).
- A generator \(g\).

Clinic A selects a private value \(a\), and Clinic B selects a private value \(b\).

Step 1: Generate public values

Clinic A calculates:

\[A=g^a\bmod p \]

Clinic B calculates:

\[B=g^b\bmod p \]

Step 2: Exchange public values

Both clinics exchange their public values through the network.

Step 3: Calculate shared secret

Clinic A calculates:

$$ K=B^a\bmod p $$

Clinic B calculates:

$$ K=A^b\bmod p $$

Both obtain the same secret because:

$$ (g^a)^b\equiv(g^b)^a\pmod p $$

Therefore, both clinics establish an identical shared secret without directly transmitting their private values.

3. Diffie–Hellman Pseudocode

Clinic A:

1. Select private key a.
2. Calculate $A=g^a\bmod p$.
3. Send public value A to Clinic B.
4. Receive public value B.
5. Calculate shared key $K=B^a\bmod p$.

Clinic B:

1. Select private key b.
2. Calculate $B=g^b\bmod p$.
3. Send public value B to Clinic A.
4. Receive public value A.
5. Calculate shared key $K=A^b\bmod p$.

Both clinics obtain the same shared secret.

4. Hand-Checkable Example

Choose:

$$ p=23,\quad g=5 $$

Clinic A selects:

$$ a=6 $$

Clinic B selects:

$$ b=15 $$

Public value calculation

Clinic A:

$$ A=5^6\bmod23=8 $$

Clinic B:

$$ B=5^{15}\bmod23=19 $$

Shared secret calculation

Clinic A:

$$ K=19^6\bmod23=2 $$

Clinic B:

$$ K=8^{15}\bmod23=2 $$

Shared secret = 2

Both clinics obtain the same shared secret successfully.

5. Larger Toy Configuration

Consider another example:

$$ p=7919,\quad g=7 $$

Clinic A selects private value:

$$ a=123 $$

Clinic B selects private value:

$$ b=456 $$

They calculate their public values:

$$ A=7^{123}\bmod7919=6833 $$

$$ B=7^{456}\bmod7919=4150 $$

Shared secret:

$$ K=4150^{123}\bmod7919 $$

$$ K=6833^{456}\bmod7919 $$

$$ \boxed{K=5785} $$

Both clinics obtain the same shared secret.

Note: These are small demonstration parameters, not secure deployment parameters.

6. Legitimate Computation vs Brute-Force Attack

The legitimate users calculate modular exponentiation using efficient square-and-multiply.

An attacker observing the public values must solve the discrete logarithm problem to recover the private exponent.

Prime size	Legitimate computation	Brute-force attack
Small prime	Very fast	Very easy
16-bit prime	Fast	Easy
32-bit prime	Fast	More difficult
64-bit prime	Fast	Extremely expensive for simple exhaustive search
2048-bit secure group	Practical with suitable implementation	Infeasible for properly selected parameters

Legitimate modular exponentiation takes approximately logarithmic time in the exponent. Brute-force discrete-log search becomes much more expensive as the group size increases.

Therefore, secure Diffie–Hellman requires appropriately large, well-selected groups.

7. Man-in-the-Middle (MITM) Attack

Unauthenticated Diffie–Hellman is vulnerable to an active man-in-the-middle attack.

An attacker intercepts the public values exchanged between the two clinics and replaces them with their own public values.

Attack process:

1. Clinic A sends public value \(A\).
2. The attacker intercepts it and sends a fake value to Clinic B.
3. Clinic B sends public value \(B\).
4. The attacker intercepts it and sends another fake value to Clinic A.
5. The attacker establishes one shared key with Clinic A and another with

Clinic B.

6. The attacker can decrypt, modify and re-encrypt messages between the clinics.

MITM message sequence

Clinic A

Attacker (MITM)

Intercepts and replaces public values

Clinic B

The attacker creates two separate shared secrets and can relay messages between the clinics.

Therefore, unauthenticated Diffie–Hellman alone cannot guarantee the identity of the communicating clinics.

8. Authentication Using Digital Signatures and Certificates

To prevent MITM attacks, digital signatures and digital certificates should be used.

- Each clinic has a public-private key pair.
- A trusted Certificate Authority (CA) issues certificates to verify clinic identities.
- Each clinic signs its Diffie–Hellman public value and relevant handshake information.
- The receiving clinic verifies the signature using the sender's certified public key.
- If verification fails, the connection is rejected.

This ensures that attackers cannot secretly replace public values without detection.

9. Key Distribution and Key Management Policy

For a network containing 25 clinics, the following policy is recommended.

1. Key Generation

Generate private keys using a cryptographically secure random-number

generator.

2. Key Storage

Store long-term private keys in secure hardware or protected key-management systems.

3. Key Distribution

Use authenticated Diffie–Hellman with digital certificates to establish session keys securely.

4. Key Rotation

Rotate long-term authentication keys periodically and establish fresh session keys for new sessions.

5. Key Revocation

Immediately revoke compromised certificates and prevent their future use.

6. Compromise Recovery

Generate new private keys, distribute updated certificates and establish new session keys after a compromise.

7. Session-Key Lifetime

Use temporary session keys for each connection. Destroy them when the session ends or when the defined lifetime expires.

10. Final Recommendation

For secure communication between 25 remote clinics, I recommend Authenticated Diffie–Hellman with digital certificates and digital signatures.

It provides:

- Secure shared-secret establishment.
- Protection against passive network observation.
- Authentication of communicating clinics.
- Protection against man-in-the-middle attacks.
- Fresh session keys for secure communication.

The established session key should be used with AES-GCM to encrypt patient

records and protect message integrity.

Conclusion:

Diffie–Hellman allows two clinics to establish a common secret key over an insecure network. However, unauthenticated Diffie–Hellman is vulnerable to man-in-the-middle attacks. Combining it with digital signatures, certificates, secure key storage, key rotation and revocation provides a stronger and more reliable solution for healthcare communication.

Q5 – ECC and Public-Key Strategy Comparison (5 Marks)

Problem Context – Constrained Devices

Remote medical devices need public-key security with manageable key size, computation and communication overhead.

Your Challenge

- Explain the mathematical basis of elliptic-curve public-key cryptography and demonstrate a toy ECC key-generation/key-agreement example.
- Compare RSA, Diffie–Hellman and ECC in key size, computational cost, bandwidth and deployment considerations.
- Explain why ECC can provide comparable security with smaller public-key material than traditional RSA-style parameters.
- Identify implementation requirements such as secure randomness, parameter validation and protected private keys.
- Recommend a public-key strategy for constrained devices and justify it using the experimental evidence from Q2–Q4.

Evidence Expected: Toy ECC example, RSA/DH/ECC comparison table and a defended recommendation.

Answer:

ECC and Public-Key Strategy Comparison (5 Marks)

1. Mathematical Basis of Elliptic-Curve Cryptography (ECC)

Elliptic-Curve Cryptography (ECC) is a public-key cryptographic technique based on the mathematical properties of elliptic curves over finite fields.

An elliptic curve is commonly represented as:

$$ y^2 \equiv x^3+ax+b \pmod p $$

where:

- a and b are curve parameters.
- p is a prime number.
- x and y are points on the curve.

ECC uses point addition and point multiplication to generate public and private keys.

Key generation:

1. Select a standard elliptic curve and a base point G.
2. Choose a random private key d.
3. Calculate the public key:

$$ Q=dG $$

Here, d is the private key, G is the base point, and Q is the public key.

Toy ECC Key-Agreement Example

Consider the elliptic curve:

$$ y^2=x^3+2x+2 \pmod {17} $$

Let the base point be $G=(5,1)$.

- Clinic A selects private key $d_A=5$.
- Clinic B selects private key $d_B=7$.

Their public keys are:

$$ Q_A=5G=(9,16) $$

$$ Q_B=7G=(0,6) $$

Both clinics calculate the shared secret:

\[K_A=5Q_B=5(7G)=35G \]

\[K_B=7Q_A=7(5G)=35G \]

Therefore, both clinics obtain the same shared point:

\[\boxed{K=(10,11)} \]

This demonstrates that two devices can establish a common secret without directly transmitting their private keys.

Note: This is a small educational example only. Real medical devices must use standardized, secure curves and cryptographic libraries.

2. Comparison of RSA, Diffie–Hellman and ECC

Feature	RSA	Diffie–Hellman (DH)	ECC
Mathematical basis	Integer factorization	Discrete logarithm	Elliptic-curve discrete logarithm
Typical public-key size	2048 bits	2048 bits (finite-field DH)	256 bits
Security level	About 112-bit at RSA-2048	Depends on group size	About 128-bit at 256-bit curves
Computation	Expensive key generation and private-key operations	Expensive modular exponentiation	Generally efficient with smaller operands
Bandwidth	Higher	Higher	Lower

Device suitability	Less suitable for highly constrained devices	Can be computationally expensive	Well suited to constrained devices
Deployment	Widely supported	Commonly used for key agreement	Common in modern mobile and embedded security

3. Why ECC Provides Comparable Security with Smaller Keys

ECC provides strong security using smaller key sizes because solving the elliptic-curve discrete logarithm problem is computationally difficult.

For example:

- RSA requires approximately a 3072-bit key to provide 128-bit security.
- ECC requires approximately a 256-bit key to provide a similar security level.

Therefore, ECC offers:

1. Smaller keys: Reduces memory requirements.
2. Lower bandwidth: Requires less data to be transmitted.
3. Efficient computation: Suitable for devices with limited processing power.
4. Lower energy consumption: Smaller operations can reduce energy usage in constrained environments.

Thus, ECC is useful for remote medical devices with limited memory, battery power and network capacity.

4. ECC Implementation Requirements

For secure ECC implementation, the following requirements must be followed:

- Secure randomness: Generate private keys using a cryptographically secure random number generator.

- Parameter validation: Use approved curves and validate received public keys and curve points.
- Private-key protection: Store private keys securely using hardware security modules or secure elements where available.
- Key management: Rotate keys when required and revoke compromised keys.
- Authentication: Use digital certificates or digital signatures to prevent man-in-the-middle attacks.
- Secure protocols: Use standardized protocols and trusted cryptographic libraries instead of designing a custom ECC system.

5. Recommended Public-Key Strategy for Remote Medical Devices

ECC is recommended for remote medical devices because it provides strong security with smaller keys, lower communication overhead and efficient computation.

The recommended approach is:

- Use ECDHE (Elliptic-Curve Diffie–Hellman Ephemeral) for secure session-key agreement.
- Use ECDSA for device authentication and digital signatures.
- Use AES-GCM for encrypting medical records after establishing the shared session key.
- Protect private keys using secure storage and rotate or revoke keys when necessary.

Justification using experimental evidence from Q2–Q4:

The earlier experiments with RSA and Diffie–Hellman demonstrate that public-

key operations require mathematical computation and that larger parameters increase processing requirements. The encryption and decryption experiments also show the importance of selecting secure algorithms and appropriate key sizes. ECC addresses the resource limitations of remote devices by achieving comparable security with smaller public-key material.

Conclusion: ECC-based key agreement with ECDHE, authentication using certificates or ECDSA, and AES-GCM for data encryption is a suitable strategy for constrained medical devices. It provides a good balance of security, speed, bandwidth efficiency and resource usage.

Q6 – Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness (7 Marks)

Problem Context – Large Confidential Document Exchange

The healthcare network must transfer large files with confidentiality, integrity and sender authenticity. The team is comparing direct public-key protection with hybrid encryption and must also consider future quantum threats.

Your Challenge

- Design a digital-signature process using hash-then-sign and explain signing and verification.
- Compare RSA, DSA/DSS and ECDSA at the conceptual level with respect to keys, signing/verification, randomness and security assumptions.
- Design hybrid encryption: encrypt the large payload with a random symmetric session key, then protect or establish that session key using RSA or Diffie–Hellman; authenticate the exchange.
- Test tampering of the file, signature and protected session key.
- Compare direct RSA encryption of large data with hybrid encryption using operation counts and file-size growth.
- Explain the basic principle of quantum key distribution/quantum cryptography and how eavesdropping can be detected.
- Explain why sufficiently capable quantum computers threaten RSA and ECC and propose a migration/readiness strategy without claiming that quantum cryptography itself replaces ordinary cryptographic key management.

Deployment Decision

Choose a practical architecture for the healthcare network and justify the choice using performance, security, key-management burden and long-term risk.

Evidence Expected: Signature flow, hybrid-encryption design, tamper tests, growth comparison, quantum-security explanation and final architecture.

DAY-4 DESIGN CHALLENGE

Answer:

Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness

1. Digital Signature Process – Hash-Then-Sign

A digital signature provides authentication, integrity and non-repudiation. It confirms the sender's identity and helps detect whether a document has been modified.

Signing process:

1. The sender prepares the confidential medical document.
2. A cryptographic hash function such as SHA-256 generates a fixed-length hash of the document.
3. The sender signs the hash using their private signing key.
4. The sender sends the document and digital signature to the receiver.

$$ h=SHA256(\text{Document}) $$

$$ S=Sign(\text{Private Key},h) $$

Verification process:

1. The receiver calculates the hash of the received document.
2. The receiver verifies the signature using the sender's public key.
3. The receiver compares the verified hash with the newly calculated hash.
4. If they match, the signature is valid and the document has not been altered.

$$ Verify(\text{Public Key},h,S)=Valid $$

Digital signature flow

Sender's medical document

SHA-256 hash generation

Sign hash using sender's private key

Send document + digital signature

Receiver verifies using sender's public key

Valid

Hashes match and signature verification succeeds.

Invalid

Document or signature has been modified, or verification fails.

2. Comparison of RSA, DSA/DSS and ECDSA

Feature	RSA Signature	DSA/DSS	ECDSA
Mathematical basis	Integer factorization	Discrete logarithm	Elliptic-curve discrete logarithm
Key size	Large	Moderate to large	Small
Signing	Uses private exponent	Uses private key and random nonce	Uses private key and random nonce
Verification	Uses public exponent	Uses public key	Uses public key
Randomness	RSA-PSS uses a salt; signing also needs secure implementation	Requires a unique, secret nonce	Requires a unique, secret nonce
Main security risk	Weak keys or poor padding	Reused or predictable nonce	Reused or predictable nonce
Main advantage	Widely supported	Designed for digital signatures	Strong security with smaller keys

Explanation:

- RSA: Supports encryption and digital signatures. RSA-PSS is the recommended modern signature padding scheme.
- DSA/DSS: Designed for digital signatures, not encryption. It depends on secure nonce generation.
- ECDSA: Uses elliptic curves to provide strong signature security with smaller keys and signatures.

If a DSA or ECDSA nonce is reused or exposed, an attacker may recover the private signing key. Therefore, secure randomness or a correctly implemented deterministic nonce-generation method is essential.

3. Hybrid Encryption Design

Hybrid encryption combines symmetric and asymmetric cryptography. It is suitable for transferring large healthcare documents because symmetric encryption is efficient for large files.

Encryption process:

1. Generate a random symmetric session key.
2. Encrypt the large medical file using AES-256-GCM.
3. Protect the session key using the receiver's RSA public key with RSA-OAEP, or establish it using authenticated Diffie–Hellman.
4. Sign the encrypted package or its authenticated metadata using the sender's private signing key.
5. Send the encrypted file, protected session key and digital signature to the receiver.

\[C=AES\text{-}GCM(K,\text{Medical File}) \]

For RSA key protection:

\[E=RSA\text{-}OAEP(\text{Receiver Public Key},K) \]

For authenticated Diffie–Hellman, both parties derive a shared session key and authenticate the exchange using certificates and digital signatures.

Decryption process:

1. Verify the sender's digital signature and certificate.
2. Recover the session key using the receiver's RSA private key, or derive it through the authenticated key exchange.
3. Decrypt the file using AES-256-GCM.
4. Verify the GCM authentication tag before accepting the document.

Hybrid encryption flow

Large medical document

Generate random AES session key

Encrypt document using AES-256-GCM

Protect session key using RSA-OAEP or authenticated DH

Authenticate package and send to receiver

4. Tampering Tests

The following tests can be performed to check the security of the hybrid encryption system.

Test	Modification made	Expected result
File tampering	Change one byte in the encrypted medical file	AES-GCM authentication fails and decryption is rejected
Signature tampering	Change or replace the digital signature	Signature verification fails
Session-key tampering	Modify the RSA-encrypted session key	Session-key recovery fails or the wrong key is produced; authenticated decryption fails
Nonce or tag tampering	Modify the AES-GCM nonce or authentication tag	Authentication fails and the file is rejected

Interpretation: These tests demonstrate that unauthorized changes to the document, signature or protected session key can be detected. The receiver must reject invalid data rather than accept partially decrypted medical records.

5. Direct RSA Encryption vs Hybrid Encryption

RSA is not suitable for directly encrypting large files because it has a limited plaintext size and requires expensive public-key operations.

Feature	Direct RSA Encryption	Hybrid Encryption
Large file encryption	Difficult and inefficient	Efficient
Operations	RSA operation for every small plaintext block if chunked	One RSA key-protection operation, plus symmetric encryption of the file
Speed	Slower	Faster
File-size growth	Many RSA ciphertext blocks create large overhead	Small session-key, nonce, tag and signature overhead

Security	Requires secure padding and careful block handling	Combines secure asymmetric key protection with authenticated symmetric encryption
Suitability	Not recommended for large documents	Recommended for healthcare file exchange

Example using a 2048-bit RSA key:

- RSA ciphertext size is 256 bytes per RSA operation.
- With RSA-OAEP and SHA-256, the maximum plaintext size is 190 bytes.
- A 10 MB file would require approximately 55,189 RSA blocks if encrypted directly in maximum-sized chunks, producing approximately 14.1 MB of RSA ciphertext, excluding other overhead.
- Hybrid encryption uses one RSA operation to protect a 32-byte AES session key. The 10 MB file is encrypted using AES-GCM, with only a small additional overhead for the nonce, authentication tag and signature.

Interpretation: Hybrid encryption reduces computational work and communication overhead. It is more practical for transferring large medical files.

6. Quantum Key Distribution and Quantum Threats

Quantum Key Distribution (QKD)

Quantum Key Distribution is a technique that allows two parties to establish a shared secret key using quantum states, such as photons.

Basic principle:

1. The sender transmits photons using randomly selected polarization bases.
2. The receiver measures the photons using randomly selected bases.
3. Both parties compare the bases they used through a public communication channel.
4. They keep the results where their bases match.
5. They compare a sample of the results to estimate whether eavesdropping has occurred.

6. If the error rate is too high, they reject the key. Otherwise, they perform error correction and privacy amplification.

How eavesdropping is detected:

According to the quantum measurement principle, an attacker who measures unknown quantum states can disturb them. This disturbance may increase the error rate, allowing the communicating parties to detect possible interception. QKD can help establish secret keys, but it does not replace authentication, endpoint security, certificates or ordinary key management.

Quantum Threat to RSA and ECC

A sufficiently powerful, fault-tolerant quantum computer could use Shor's algorithm to solve the mathematical problems underlying RSA and conventional ECC.

- RSA security depends on the difficulty of factoring large integers.
- ECC security depends on the difficulty of solving the elliptic-curve discrete logarithm problem.
- A sufficiently capable quantum computer could break both systems.

Grover's algorithm could also speed up brute-force searches against symmetric keys. Using larger symmetric keys, such as AES-256, provides additional protection.

Quantum-Readiness Strategy

The healthcare network should:

1. Identify systems that currently use RSA and ECC.
2. Protect confidential medical records that may need to remain secret for many years.
3. Adopt standardized post-quantum cryptographic algorithms, such as ML-KEM for key establishment and ML-DSA for digital signatures.
4. Test hybrid classical and post-quantum key-establishment methods during migration.
5. Maintain secure key storage, rotation, revocation and backup procedures.

6. Update devices and protocols so that algorithms can be replaced when necessary.

7. Final Deployment Decision

Recommended architecture

The healthcare network should adopt hybrid encryption with digital signatures and post-quantum migration readiness.

The proposed architecture is:

- AES-256-GCM: Encrypt large medical documents and provide data integrity.
- RSA-OAEP or authenticated ECDHE: Protect or establish the symmetric session key during the current deployment.
- RSA-PSS or ECDSA: Authenticate the sender using digital signatures.
- Digital certificates: Verify the identities of clinics and medical devices.
- Secure key management: Protect private keys, rotate keys and revoke compromised credentials.
- Post-quantum migration: Introduce standardized ML-KEM and ML-DSA as supported devices and systems are upgraded.

Justification:

Hybrid encryption is faster and more efficient than directly encrypting large files using RSA. Digital signatures detect document modifications and verify the sender's identity. Secure key management reduces the risk of key theft and misuse. Post-quantum migration helps protect the healthcare network against future quantum-computing threats.

Conclusion: Hybrid encryption using AES-256-GCM, authenticated key establishment and digital signatures is the most practical approach for large confidential healthcare document exchange. The network should also prepare to migrate from RSA and ECC to standardized post-quantum cryptography.

Q7 – Diagnose, Redesign and Validate the Complete Cryptographic

Strategy (11 Marks)

Review the evidence from Q1–Q6 and identify one significant computational or security limitation that becomes important as message volume, key size, number of clinics or attacker capability increases.

A. Diagnose the Bottleneck or Weakness (2 Marks)

- Identify the operation or missing security property responsible for the limitation.
- Explain how its cost or risk changes with input/key size or repeated use.
- Use measured operation counts or attack evidence from earlier tasks.

B. Define the Design Objective (1 Mark)

State a measurable redesign objective, such as reducing public-key operations, preventing a man-in-the-middle, reducing key-management exposure, detecting weak keys, or improving secure bulk-data handling.

C. Design the Revised Strategy (3 Marks)

Develop a revised architecture using the appropriate combination of symmetric encryption, secure modes, RSA/DH/ECC, digital signatures, key management and hybrid encryption. Explain the mathematical/security property exploited, operations reduced, additional state or keys required, and new trade-offs.

D. Integrate Quantum Readiness (1 Mark)

Identify which RSA/ECC dependencies are threatened by future quantum computing and define a migration approach. Explain separately what quantum key distribution can contribute and what it cannot replace.

E. Predict, Implement and Validate (2 Marks)

Predict the improvement before implementation. Compare original and redesigned solutions using operation counts, attack steps, key-management events, verification results and execution time where useful. Verify that the redesign preserves required functionality.

F. Defend the Final Design (2 Marks)

Defend the final decision against the original alternatives. Discuss residual risks, deployment cost, usability, key compromise, MITM attacks, weak keys, large-file performance and long-term quantum risk.

Evidence Expected: Diagnosis, measurable objective, revised architecture/protocol, prediction, comparable experimental evidence and final engineering defence.

Answer:

A. Diagnose the Bottleneck or Weakness (2 Marks)

The major limitation in the existing healthcare cryptographic system is the high computational cost of repeated public-key operations and the risk of insecure key management when transferring large medical files between multiple clinics.

Identified problems:

1. Direct RSA encryption of large files requires many public-key operations, increasing processing time and ciphertext size.
2. Repeated use of the same encryption key increases the impact of key compromise.
3. Unauthenticated Diffie–Hellman key exchange is vulnerable to man-in-the-middle attacks.
4. RSA and ECC are vulnerable to sufficiently powerful future quantum computers.

Experimental evidence:

For a 10 MB file using 2048-bit RSA with OAEP and SHA-256:

- Maximum plaintext per RSA operation = 190 bytes.
- Number of RSA operations required = approximately 55,189.
- Each RSA ciphertext block occupies 256 bytes.
- Total ciphertext size is approximately 14.1 MB.

In contrast, hybrid encryption requires only one RSA operation to protect a 32-byte AES session key. AES-GCM handles the entire file efficiently.

Therefore, repeated RSA operations are a major performance bottleneck.

B. Define the Design Objective (1 Mark)

The objective is to redesign the cryptographic system to:

- Reduce public-key operations for large-file transfers.
- Maintain confidentiality, integrity and sender authentication.
- Prevent man-in-the-middle attacks.

- Reduce the risk of key reuse and key compromise.
- Support multiple clinics with secure key management.
- Prepare for future quantum-computing threats.

Measurable objective: Replace thousands of RSA encryption operations per large-file transfer with one session-key protection operation, while ensuring that file tampering and invalid signatures are rejected.

C. Design the Revised Strategy (3 Marks)

The revised architecture combines symmetric encryption, authenticated public-key key establishment, digital signatures and secure key management.

Proposed architecture

Revised healthcare cryptographic architecture

1. Sender authentication

Digital certificate and signature verification

2. Secure key establishment

Authenticated ECDHE or RSA-OAEP during compatible deployments

3. Session-key generation

Fresh random AES-256 key for each transfer

4. Bulk-data encryption

AES-256-GCM with a unique nonce

5. Secure transmission

Encrypted medical file + authentication tag + signed metadata

6. Receiver verification

Verify signature, authenticate ciphertext and decrypt the file

Explanation of the revised strategy

1. Symmetric encryption

AES-256-GCM encrypts large medical documents and provides confidentiality and integrity.

\[C=AES\text{-}GCM(K,P,Nonce) \]

A fresh session key is generated for each transfer, and a unique nonce is used with

that key. Nonce reuse with AES-GCM must never occur.

2. Public-key key establishment

Authenticated ECDHE establishes a shared session key between clinics. Certificates and digital signatures authenticate the exchanged public values and prevent man-in-the-middle attacks.

RSA-OAEP can be used to protect session keys in compatible systems, but authenticated ECDHE is preferred for modern session establishment.

3. Digital signatures

The sender signs the document hash or authenticated transfer metadata using RSA-PSS or ECDSA. The receiver verifies the signature using the sender's public key.

4. Key management

- Generate keys using secure random-number generators.
- Maintain separate keys for different clinics and sessions.
- Store private keys in secure hardware or a protected key-management system.
- Rotate and revoke keys when necessary.
- Maintain audit logs for key usage and access.
- Destroy temporary session keys after the transfer is completed.

Mathematical/security property: ECC uses the difficulty of the elliptic-curve discrete logarithm problem to establish shared secrets efficiently. AES-GCM uses symmetric encryption with an authentication tag to protect large amounts of data.

Trade-offs: The revised system requires certificate management, secure key storage, nonce tracking and additional authentication steps. However, it reduces public-key computation and limits the damage caused by individual session-key compromise.

D. Integrate Quantum Readiness (1 Mark)

RSA and ECC depend on mathematical problems that sufficiently capable quantum computers could solve using Shor's algorithm.

Therefore, the healthcare network should prepare to replace vulnerable public-key mechanisms.

Migration strategy:

- Use ML-KEM for post-quantum key establishment.
- Use ML-DSA for post-quantum digital signatures.
- Test hybrid classical and post-quantum key establishment.
- Upgrade devices, certificates and communication protocols gradually.
- Protect long-term confidential medical records against future decryption risks.

Role of QKD: Quantum Key Distribution can help two parties establish shared secret keys and detect possible eavesdropping through disturbances in quantum measurements. However, it requires specialized communication infrastructure and does not replace endpoint security, authentication, digital signatures or key management.

E. Predict, Implement and Validate (2 Marks)

1. Predicted improvement

The redesigned system should reduce public-key operations, improve large-file transfer efficiency and detect unauthorized modifications.

Measurement	Original direct RSA	Redesigned hybrid system
10 MB file encryption	About 55,189 RSA operations	One RSA session-key protection operation, or one authenticated key exchange
Bulk encryption	RSA	AES-256-GCM
Ciphertext size	About 14.1 MB	Approximately 10 MB plus small metadata and authentication overhead
File tampering detection	Depends on additional integrity protection	AES-GCM authentication tag
Sender authentication	Requires a separate signature mechanism	Digital signature and certificate
Key usage	Can involve repeated long-term key use	Fresh session key for each transfer

2. Validation tests

Test	Expected result
Encrypt and decrypt a medical file	Original file is recovered correctly
Modify encrypted file	Authentication fails
Modify digital signature	Signature verification fails
Modify protected session key	Key recovery or authenticated decryption fails
Use an invalid public key	Parameter or certificate validation rejects it
Attempt man-in-the-middle attack	Authentication prevents acceptance of the attacker's key
Repeat transfer with a fresh session key	New transfer uses independent key material

3. Experimental validation

Measure the following for both systems:

- Number of public-key operations.
- Encryption and decryption execution time.
- Ciphertext size.
- Successful and failed signature-verification results.
- Number of key-generation, rotation and revocation events.

The prediction is that hybrid encryption will significantly reduce public-key processing time for large files while preserving confidentiality and integrity. The system should also reject tampered files and invalid signatures.

Actual execution times and test results should be recorded from the implementation rather than assumed.

F. Defend the Final Design (2 Marks)

The revised hybrid architecture is preferred over direct RSA encryption, unauthenticated Diffie–Hellman and single-key encryption systems.

Advantages:

1. Performance: AES-GCM efficiently encrypts large medical files, reducing

expensive public-key operations.

2. Security: Digital signatures provide sender authentication, while authenticated encryption detects data modification.
3. MITM protection: Certificate validation and authenticated key exchange prevent unauthorized parties from replacing public keys.
4. Key management: Fresh session keys reduce the impact of individual key compromise.
5. Scalability: Separate clinic identities and controlled key management support communication among multiple healthcare centres.
6. Quantum readiness: Post-quantum algorithms provide a migration path against future quantum attacks.

Residual risks and limitations:

- Stolen private keys can allow attackers to impersonate legitimate clinics.
- Weak random-number generation can expose session keys.
- Incorrect AES-GCM nonce management can seriously weaken security.
- Certificate expiration and revocation must be handled correctly.
- Post-quantum algorithms may require more memory, bandwidth and processing power on constrained devices.
- Secure implementation, regular updates and staff training are still necessary.

Final engineering decision:

The healthcare network should adopt AES-256-GCM for bulk-data encryption, authenticated ECDHE for session-key establishment, digital signatures for authentication, and centralized secure key management. It should gradually introduce ML-KEM and ML-DSA to prepare for quantum threats.

Conclusion: The redesigned architecture improves performance, confidentiality, integrity, authentication and scalability compared with direct public-key encryption. It provides a practical security solution for large medical document transfers while supporting future cryptographic upgrades.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Symmetric cryptosystems, DES/3DES/Blowfish/RC5, block/stream ciphers and modes	7
Q2	Number theory, modular arithmetic and prime selection	6
Q3	RSA encryption/decryption, attacks, weak keys and padding	7
Q4	Diffie–Hellman, MITM authentication and key management	7
Q5	ECC and RSA/DH/ECC comparison	5
Q6	Digital signatures, hybrid encryption and quantum readiness	7
Q7	Day-4 integrated diagnose, redesign and validation	11
TOTAL		50

Submission Requirements

Submit your assignment report containing Q1–Q7 evidence, source code for student-implemented core algorithms, raw experimental data, tables/graphs and protocol/message-sequence diagrams. Libraries may be used for cross-checking standard primitives, but the mathematical core and experiments must be your own.

All attack demonstrations must use only student-created toy parameters and test data. Do not use classroom code to protect real patient data.

Assessment Verification

Students may be asked to explain algorithm design, source code, test data, mathematical predictions, experimental results, security analysis and the Day-4 redesign during a brief verification/viva. Both the reasoning and the evidence must be understood by the student.

- Algorithmic and protocol design
- Source code
- Test-data and key-set design
- Experimental results
- Predictions
- Security analysis
- Design Challenge

during a brief verification discussion/viva. The student should therefore be able to explain and defend the evidence submitted as their own work.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook cryptosystems.

It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Attack → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by your own mathematical reasoning and experimental evidence.